

STANDAR DOKUMENTASI & RENCANA PENGUJIAN (TEST PLAN)

Panduan Terintegrasi: DFD Level 1, Sequence Diagram, dan Testing

1. STANDAR INSTALLATION MANUAL

Mahasiswa wajib mendetailkan proses setup aplikasi agar dapat dijalankan oleh penguji atau pengguna lain.

1.1 System Requirements (Spesifikasi Sistem)

Untuk menjalankan aplikasi mycafe dengan optimal, sistem disarankan memenuhi kriteria minimum berikut.

A. System Requirements:

Processor: Intel Core i3 atau setara
RAM: Minimal 4 GB (disarankan 8 GB)
Storage: Minimal 2 GB ruang kosong
Koneksi internet untuk instalasi dependency

B. Environment Setup:

Operating System: Windows 10/11, Linux, atau macOS
PHP: Versi 8.3 atau lebih baru
Composer: Versi 2.x atau lebih baru
Laravel: Versi 13
MySQL: Versi 8.0 atau lebih baru
Web Browser: Google Chrome, Microsoft Edge, atau Mozilla Firefox

1.2 Environment Setup & Konfigurasi File .env

Aplikasi mycafe dikembangkan menggunakan framework Laravel 13 dengan database MySQL. Konfigurasi aplikasi disimpan pada file .env yang berada di direktori utama proyek. File tersebut digunakan untuk mengatur berbagai parameter penting seperti koneksi database, application key, konfigurasi mail, serta pengaturan lingkungan aplikasi lainnya agar sistem dapat berjalan dengan baik dan aman.

A. Konfigurasi Backend

Buat file bernama .env di dalam root direktori backend dan masukkan konfigurasi berikut:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=mycafe
```

DB_USERNAME=root

DB_PASSWORD=

B. Konfigurasi Frontend (frontend/.env)

Buat file bernama .env di dalam root direktori frontend (frontend/) dan masukkan konfigurasi berikut:

VITE_APP_NAME="\${APP_NAME}"

1.3 Database Deployment (Migrasi Laravel dengan Database)

1. Membuat Database MySQL

Buat database baru pada MySQL menggunakan perintah berikut:

```
CREATE DATABASE mycafe_db;
```

2. Konfigurasi Koneksi Database

Buka file .env pada root project Laravel, kemudian sesuaikan konfigurasi database:

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=mycafe

DB_USERNAME=root

DB_PASSWORD=

3. Generate Application Key

Jalankan perintah berikut untuk membuat application key Laravel:

```
php artisan key:generate
```

4. Menjalankan Migrasi Database

Jalankan migrasi untuk membuat seluruh tabel yang dibutuhkan aplikasi:

```
php artisan migrate
```

5. Menjalankan Seeder (Opsional)

Jika aplikasi menyediakan data awal (*dummy data* atau *master data*), jalankan perintah:

```
php artisan migrate --seed
```

1.4 Execution (Cara Menjalankan Aplikasi)

A. Cara Menjalankan Backend Server

- 1. Buka terminal baru di direktori / cd mycafe
- 2. Jalankan perintah berikut untuk mengaktifkan server backend:

```
php artisan serve
```

Server backend akan berjalan aktif di alamat <http://127.0.0.1:8000>

B. Cara Menjalankan Frontend Server

- 1. Buka terminal baru di direktori frontend/.
- 2. Install dependensi frontend jika belum pernah dijalankan sebelumnya:

```
composer install
```

- 3. Jalankan server development lokal laravel:

```
php artisan serve
```

2. BACKEND TEST PLAN (Berdasarkan Sequence Diagram)

Pengujian backend dilakukan menggunakan Postman dengan aturan **Traceability** sebagai berikut:

Sequence Diagram	Daftar Method	Return Value
Seq1_Login	login(LoginRequest) ensureIsNotRateLimited() Auth::attempt(email, password) RateLimiter::clear()	Status : 200 OK/302 FOUND
Seq2_KelolaMenu		
Seq2_Buka Form Tambah Menu	create() Category::orderBy('cat_name', 'asc')->get()	View (HTML): admin.item.create (Membawa data categories untuk opsi dropdown)
Seq2_Simpan Menu Baru	store(Request)	Redirect (302 Found):

	<pre> \$request->validate() \$request->hasFile('img') Item::create() </pre>	<pre> { "Item created successfully." } </pre>
Seq2_Update Data Menu	<pre> update(Request, \$id) \$request->validate() Item::findOrFail(\$id) \$item->update() </pre>	<pre> { "Item updated successfully." } </pre>
Seq2_Hapus Menu	<pre> destroy(\$id) Item::findOrFail(\$id) \$item->delete() </pre>	Redirect (302 Found): <pre> { "Item deleted successfully." } </pre>
Seq2_Ubah Status Aktif Menu	<pre> updateStatus(\$id) Item::findOrFail(\$id) \$item->save() </pre>	Redirect (302 Found): <pre> { "Item status updated successfully." } </pre>
Seq3_Pembuatan Pesanan dan Pembayaran		
Seq3_TambahKeranjang	<pre> addToCart(Request) Item::find(menuId) Session::get('cart') Session::put('cart') </pre>	<pre> { "status": "success", "message": "Berhasil ditambahkan ke keranjang", "cart": {...} } </pre>
Seq3_Update Jumlah Item	<pre> updateCart(Request) Session::get('cart') Session::put('cart') Session::flash('success') </pre>	<pre> {"success": true} </pre>

Seq3_Simpan Pesanan	storeOrder(Request) Validator::make() User::firstOrCreate() Order::create() OrderItem::create() Session::forget('cart')	Redirect (302 Found): { "Pesanan berhasil dibuat" }
Seq3_Simpan Pesanan (QRIS - Midtrans)	storeOrder(Request) \Midtrans\Snap::getSnapToken()	Status: 200 OK (JSON) { "status": "success", "snap_token": "TOKEN_MIDTRANS_...", "order_code": "ORD-..." }
Seq4_KelolaPengguna		
Seq4_Simpan Staf Baru	store(Request) \$request->validate() bcrypt() User::create()	Redirect (302 Found): { "User created successfully." }
Seq4_Update Data Staf	update(Request, User) \$request->validate() Hash::check() \$user->update()	Redirect (302 Found): { "User updated successfully." }
Seq4_Hapus Akun Staf	destroy(\$id) User::findOrFail(\$id) \$user->delete()	Redirect (302 Found): { "User deleted successfully" }.
Seq5_KelolaRole		

Seq5_Simpan Role Baru	store(Request) \$request->validate() Role::create()	Redirect (302 Found): { "Role created successfully." }
Seq5_Update Data Role	update(Request, \$id) \$request->validate() Role::findOrFail(\$id) \$role->update()	Redirect (302 Found): { "Role updated successfully." }
Seq5_Hapus Data Role	destroy(\$id) Role::findOrFail(\$id) \$role->delete()	Redirect (302 Found): { "Role deleted successfully." }
Seq6_KelolaKategori		
Seq6_Simpan Kategori Baru	store(Request) \$request->validate() Category::create()	Redirect (302 Found): { "Category created successfully." }
Seq6_Update Data Kategori	update(Request, \$id) \$request->validate() Category::findOrFail(\$id) \$category->update()	Redirect (302 Found): { "Category updated successfully." }
Seq6_Hapus Data Kategori	destroy(\$id) Category::findOrFail(\$id) \$category->delete()	Redirect (302 Found): { "Category deleted successfully." }

3. USABILITY TEST PLAN (Berdasarkan DFD Level 1)

Pengujian ini ditujukan untuk user non-teknis guna memvalidasi alur bisnis yang didefinisikan pada DFD Level 1.

3.1 Skenario Uji (User Task Scenarios)

Memvalidasi apakah pengguna dapat menyelesaikan alur bisnis sistem pemesanan kafe sesuai dengan DFD Level 1 tanpa bantuan developer.

Task	Nama	Tujuan	Skenario	Langkah pengguna	Expected Result
1	Login dan Autentikasi (Proses 1.0)	Memastikan pengguna dapat masuk ke sistem menggunakan akun yang valid.	Pengguna diberikan akun yang telah terdaftar dan diminta melakukan login.	-Membuka halaman login. -Mengisi -username/email. -Mengisi password. -Menekan tombol Login.	-Pengguna berhasil masuk ke dashboard sesuai role. -Sistem menampilkan halaman utama.
2	Kelola Pengguna (Proses 2.0)	Memastikan Admin dapat menambahkan pengguna baru.	Admin diminta membuat akun baru untuk Kasir.	-Membuka menu Kelola Pengguna. -Memilih Tambah Pengguna. -Mengisi data pengguna. -Memilih role. -Menyimpan data.	-Data pengguna tersimpan. -Pengguna baru muncul pada daftar pengguna.
3	Kelola Menu dan Kategori (Proses 3.0)	Memastikan Admin dapat menambahkan menu baru.	Admin diminta menambahkan menu "Cappuccino" ke kategori Minuman.	-Membuka menu Kelola Menu. -Memilih Tambah Menu. -Mengisi nama menu. -Memilih kategori. -Mengisi harga. -Menyimpan data.	-Menu baru tersimpan. -Menu tampil pada daftar menu.
4	Membuat Pesanan (Proses 4.0)	Memastikan pelanggan dapat melakukan pemesanan.	Pelanggan diminta memesan satu menu yang tersedia.	-Membuka halaman menu. -Memilih item menu. -Menambahkan ke pesanan. -Melakukan checkout.	-Pesanan berhasil dibuat. -Data pesanan tersimpan dalam sistem.
5	Verifikasi Pembayaran	Memastikan Kasir dapat memverifikasi	Kasir menerima pembayaran dari	-Membuka daftar pesanan.	Status pesanan

	(Proses 5.0)	pembayaran pelanggan.	pelanggan dan melakukan konfirmasi pembayaran.	-Memilih pesanan pelanggan. -Memverifikasi pembayaran. -Menyimpan status pembayaran.	berubah menjadi "Lunas" atau "Dibayar".
6	Melihat Laporan Pesanan (Proses 6.0)	Memastikan Barista atau Kasir dapat melihat daftar pesanan yang masuk.	Pengguna diminta melihat daftar pesanan yang telah dibuat pelanggan.	-Membuka menu Laporan Pesanan. -Melihat daftar pesanan. -Mencari pesanan tertentu.	-Daftar pesanan berhasil ditampilkan. -Informasi pesanan dapat ditemukan.

- **Target Peserta:** Minimal 3-5 orang yang merepresentasikan end-user (bukan developer).
- **Metrik:**
 - *Completion Rate:* Apakah user berhasil menyelesaikan alur sesuai DFD?
 - *Error Rate:* Berapa kali user mengalami kendala navigasi?

4. MATRIKS PENYELARASAN (Traceability Matrix)

ID DFD 1	Nama Sequence Diagram	Method Backend (Postman)	Skenario Usability
1.0	SD_Autentikasi_Login	LoginRequest->authenticate() Payload: email, password	Menampilkan Form Login, validasi kesalahan input, dan pembatasan percobaan login (Rate Limiting).
2.0	SD_Kelola Pengguna	UserController (store, update, destroy) RoleController (store, update, destroy) Menyimpan ke D1: users & D2: roles	Admin mengelola data akun staf dan mengatur level otoritas (hak akses) masing-masing staf kafe.

ID DFD 1	Nama Sequence Diagram	Method Backend (Postman)	Skenario Usability
3.0	SD_Kelola Menu & Kategori	itemController (store, update, destroy) CategoryController (store, update, destroy) Menyimpan ke D3: categories & D4: items	Admin menambah data makanan/minuman baru, mengunggah foto menu, serta mengatur pengelompokan kategorinya.
4.0	SD_Buat Pesanan	MenuController->addToCart() MenuController->storeOrder() Membaca D4: items & menyimpan ke D5: orders	Pelanggan/Kasir memilih item menu dari keranjang belanja session, lalu backend membuat kode order_code baru.
5.0	SD_Verifikasi Pembayaran	MenuController->storeOrder() (Bagian API Midtrans) MenuController->checkoutSuccess() Menghitung D6: order_items & update status ke D5: orders	Sistem memproses transaksi. Jika via QRIS, backend meminta token ke Midtrans dan otomatis mengubah status menjadi 'settlement' jika sukses.
6.0	SD_Laporan Pesanan	OrderController->index() OrderController->updateStatus() Membaca data dari D5: orders	Barista/Kasir memantau antrean pesanan masuk, serta memperbarui status proses hidangan ('cooked').